Sprint 0: Storage & Crypto Primitives

- Setup SQLDelight + SQLCipher encrypted database engine
- Define base tables: CONTACTS, SPACES, ROOMS, ROOM_MEMBERS, MESSAGES, OUTBOX, FILE_TRANSFERS, DEDUP, CRYPTO_SESSIONS
- E2EE: Implement Ed25519 Identity Keys & X25519 DH Primitives
- Implement envelope protection interfaces: protect/open with Lazy Decryption
- Acceptance: App boots, unlocks DB with hardware keystore, signs mock payload

Sprint 1: Router MVP + 1-on-1 E2EE

- E2EE: Implement X3DH Handshake & Double Ratchet session state
- Implement Router core: transport selection WebRTC-first then Tor fallback
- Wire Router to SQLDelight DEDUP table keyed by packetId + source
- Implement ACK send/receive state transitions with OUTBOX timeout retry
- Acceptance: offline 1-on-1 text delivery works with true Double Ratchet encryption

Sprint 2: Sync + Two-Tiered DAG

- Implement Vector Clocks for Two-Tiered DAGs: Global Control vs Room Chats
- Implement non-blocking gap tracker and 'is_orphaned' DB/UI state
- Implement gap request protocol and ancestor streaming loop
- Implement PENDING_SYNCS table behavior for offline peers
- Acceptance: out-of-order message arrives, renders with UI warning, and updates after gap closure

Sprint 3: Persistence + Boot Recovery

- Implement Boot Orchestrator state machine for OUTBOX, FILE_TRANSFERS, and CAUSAL_HOLD
- Implement relay retention and LRU eviction policy hooks for Tor Swarm
- Implement DB migration strategy for protocol state schema versioning
- Acceptance: restarting the app during active routing gracefully recovers state via Boot Orchestrator

Sprint 4: Routing, RBAC, & Firewall

- Implement PING / PONG heartbeat scheduler with Vector Clock payload
- Implement peer uptime math & Tor Swarm selection algorithm targeting top 3 peers
- Implement 'Add User', 'Grant Admin', & 'Ban' Global System Events
- Implement socket-level Decentralized Firewall dropping Tor connections for BANNED contacts
- Acceptance: Banned device is instantly physically exiled from the Tor mesh socket listener

Sprint 5: Large Files & Group Sender Keys

- E2EE: Implement Sender Key Megolm-style pattern for group payload encryption
- E2EE: Wire Admin 'BAN' events to trigger Perfect Forward Secrecy key rotations
- Implement FileEnvelope lifecycle: OFFER CHUNK ACK COMPLETE CANCEL
- Implement chunk scheduler with maxInFlightChunks and missing-chunk retransmit
- Acceptance: large group transfer fans out tiny keys and resumes correctly

Sprint 6: WebRTC Resilience + AV

- Implement offer retry queues for voice and file with distinct policies
- Implement NAT-failure branching with passive wait for file path
- Expand AV lifecycle: accept reject control updates teardown
- Implement 4-5 person hard cap for P2P WebRTC group calls
- Acceptance: retry and fallback behavior matches policy under network churn

Sprint 7: Hardening + Release Readiness

- Build full scenario matrix tests: tor-only webrtc-first offline relay restart
- Add load and soak tests for 10 to 20 peer target profile
- Implement Independent Watchdog Daemon & Clearnet Github fallback
- E2EE: Audit cryptographic boundaries and wipe-from-RAM behavior
- Acceptance: release candidate passes resilience, recovery, and crypto gates

Done criteria

- Each sprint exits with at least one end-to-end integration test
- No sprint closes without explicit rollback and recovery behavior
- Security-sensitive paths include negative tests before feature marked complete